

Guía de Uso: Evaluador (Grader) - Examen 2:

Este evaluador automático (Grader) está diseñado para probar sus soluciones algorítmicas de forma local antes de la entrega final. El programa construirá grafos ocultos, los inyectará en su código y validará si los algoritmos implementados usando su propia biblioteca devuelven los resultados correctos.

Requisitos Previos

El evaluador soporta **C++**, **Python** y **Rust**. Dependiendo del lenguaje que elijan para resolver los problemas, asegúrense de tener instalado el compilador o entorno correspondiente en su sistema:

- **Python:** Python 3 instalado y agregado al PATH (`python` o `python3`).
- **C++:** GCC (`g++`) en Linux/Windows.
- **Rust:** Compilador de Rust y gestor de paquetes Cargo instalados (`rustc` y `cargo`).

Estructura de los Problemas

El evaluador soporta la validación de los 4 problemas del examen. Al ejecutarlo, pueden elegir evaluar un problema en específico o todos a la vez:

1. `ex1` (El Evento Principal)
 2. `ex2` (El Atraco Fantasma)
 3. `ex3` (Protocolo de Triage de Red)
 4. `ex4` (Orquestador de Compilación)
- `all` (Ejecuta todas las pruebas de los 4 problemas de forma secuencial).

Instrucciones de Uso

Paso 1: Configurar su Entorno (¡MUY IMPORTANTE!)

Han recibido los archivos divididos en las carpetas `linux` y `windows` dependiendo de su sistema operativo. Elijan su carpeta y el lenguaje en el que van a trabajar.

Antes de escribir una sola línea de código, **deben copiar los archivos de la biblioteca de grafos que construyeron durante las clases y en sus casa** en la misma carpeta donde está la plantilla de su lenguaje:

- Si usan **C++**: Peguen `graph.hpp` y `graph_algorithms.hpp` dentro de la carpeta `cpp/`, justo al lado de `template.cpp`.
- Si usan **Python**: Peguen `graph.py` y `graph_algorithms.py` dentro de la carpeta `python/`, justo al lado de `template.py`.
- Si usan **Rust**: Peguen `graph.rs` y `graph_algorithms.rs` dentro de la carpeta `rust/`, junto a `template.rs` y el archivo `Cargo.toml`.

Paso 2: Escribir el código

Abran el archivo de plantilla del lenguaje de su preferencia (ej. `template.cpp`). **IMPORTANTE:** Escriban su lógica **únicamente** reemplazando los `TODO` dentro de las funciones indicadas. No modifiquen la parte inferior del código (`main` o bloque I/O boilerplate) que lee los archivos y escribe en `solution.txt`. El evaluador depende de esa estructura para comunicarse con su código.

Paso 3: Ejecutar el evaluador

Abran su terminal o línea de comandos, naveguen hasta la raíz de la carpeta de su sistema operativo (donde se encuentra `grader.exe` o `grader.out`) y ejecuten el comando con la siguiente estructura:

En Windows:

```
.\grader.exe <problema> <ruta_al_archivo_template>
```

En Linux: Primero, denle permisos de ejecución al evaluador (solo se hace una vez):

```
chmod +x grader.out
```

Luego, ejecútenlo:

```
./grader.out <problema> <ruta_al_archivo_template>
```

Ejemplos de Ejecución

Supongamos que están programando en C++ y quieren probar el ejercicio 1:

En Windows:

```
.\grader.exe ex1 cpp\template.cpp
```

Supongamos que están en Linux usando Python y quieren probar todos los ejercicios de una vez para ver su nota final:

En Linux:

```
./grader.out all python/template.py
```

(Nota: El evaluador compilará automáticamente el código por ustedes de forma transparente o invocará a Cargo en el caso de Rust).

Entendiendo los Resultados

Al ejecutar el evaluador, verán en la consola el progreso de los 6 casos de prueba (incluyendo casos imposibles y pruebas de estrés a gran escala) por cada problema:

- **[PASS]:** Su programa generó la respuesta correcta para ese caso.
- **[FAIL]:** Su programa falló la prueba o devolvió una respuesta incorrecta. Revisen la topología de su solución.

Notas Importantes

- **Archivos temporales:** Durante la ejecución, el evaluador creará archivos como `test_graph.txt`, `params.txt` y `solution.txt`. No se preocupen por ellos, el evaluador los borrará automáticamente al terminar.
- **Bucles infinitos:** Si su código tiene un bucle infinito en algún recorrido DFS/BFS, el evaluador se quedará "colgado" esperando. Si esto pasa, cancelen la ejecución con `Ctrl + C` y revisen su lógica. Si quedan archivos `.txt` residuales tras cancelar, bórrenlos manualmente.

¡Mucho éxito demostrando sus habilidades como ingenieros!